



TRAINING PROGRAMME



Lesson 4

SOLID Principles ++

*"Everything should be made as simple as possible,
but not simpler."— Albert Einstein*

SOLID++: Single Responsibility

A *subsystem*, *module*, *class*, or even a *function*, should *not have more than one* reason to change.

*“Write programs that do one thing and do it well.
Write programs to work together.”*

Peter H. Salus, ‘A Quarter-Century of Unix’ (1994)



SOLID++: Single Responsibility

```
public class Car{  
    public int CurrentMileage(){...}  
    public void TravelTo(Location location){...}  
    public void Save(){...}  
}
```



SOLID++: Open/Closed

A good module structure should be both *open* and *closed*:

→ *open for extension,*

→ *closed for modification.*

“Closed with respect to X” means that clients are not affected if X changes.

It means you can add a new feature just adding new code (Open) and without changing any existing code (Closed).



SOLID++: Open/Closed

```
public class CarEngineStatusReportController{  
    public View DisplayEngineStatusReport(){  
        var webView = new CarEngineWebView();  
        webView.FillWith(carEngineViewModel);  
        return webView;  
    }  
  
    public View PrintEngineStatusReport(){  
        var printView = new CarEnginePrintView();  
        printView.FillWith(carEngineViewModel);  
        return printView;  
    }  
}
```



SOLID++: Open/Closed

```
public class CarEngineStatusReportController{
    public View DisplayEngineStatusReport(){
        var webView = new CarEngineWebView();
        webView.FillWith(carEngineViewModel);
        return webView;
    }

    public View PrintEngineStatusReport(){
        var printView = new CarEnginePrintView();
        printView.FillWith(carEngineViewModel);
        return printView;
    }

    public View AlternativeDisplayEngineStatusReport(){
        var webView = new EnhancedAccessCarEngineWebView();
        webView.FillWith(carEngineViewModel);
        return printView;
    }
}
```



SOLID++: Open/Closed

```
public class CarEngineStatusReportController{
    IDisplayEngineStatusReport _carEngineView;

    public CarEngineStatusReport(IDisplayEngineStatusReport carEngineView)
    {
        _carEngineView = carEngineView;
    }

    public View EngineStatusReport()
    {
        _carEngineView.FillWith(carEngineViewModel);
        return _carEngineView;
    }
}
```



SOLID++: Open/Closed

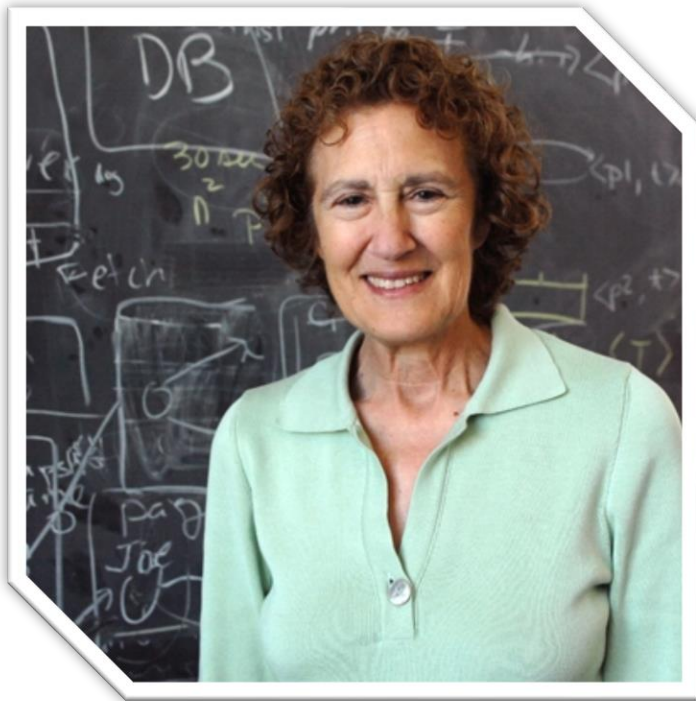
```
public class CarEngineWebView : IDisplayEngineStatusReport
{
    void FillWith(CarEngineViewModel viewModel) {
        ...
    }
}
```

```
public class CarEnginePrintView : IDisplayEngineStatusReport
{
    void FillWith(CarEngineViewModel viewModel) {
        ...
    }
}
```

```
public class EnhancedAccessCarEngineWebView : IDisplayEngineStatusReport
{
    void FillWith(CarEngineViewModel viewModel) {
        ...
    }
}
```



SOLID++: Liskov substitution



Barbara Liskov



SOLID++: Liskov substitution

Derived classes should keep promises made by parent types (interfaces, abstract or parent classes). For example, ***classes that implement some interface should keep the promises made by that interface.*** (No refused bequest code smell!!!)

It's about right usage of inheritance in OO: it has to be seen from the perspective of ***exposed behaviour***, not the internal structure or the abstract meaning.

Inheritance is a “Is a kind of” relationship (e.g. GrillOven is a kind of Oven) and that subclasses are exposing a more specific behaviour than the parent type (e.g. cooking by grilling instead of just cooking).

A quick way to check for violations: **you must be able to replace the subclass EVERYWHERE the parent class is used, without breaking the meaning of the code.** For this reason, **favour composition over inheritance** (e.g. Player “Has a” wallet instead of Player “Is a Kind of” a wallet, to get access to money).



SOLID++: Liskov substitution

```
class Oven {  
    void Cook()  
}  
  
class GrillOven : Oven{  
    void Cook(){...}  
}  
  
class FanOven : Oven{  
    void Cook(){...}  
}  
  
class Microwave : Oven{  
    void CookMicrowaving(){...}  
    void Cook(){throw ...}  
}
```

```
class Chef {  
    public void Cook(Oven oven){  
        switch(typeof(oven))  
        case Microwave m:  
            //don't call m.Cook()!!!  
            m.CookMicrowaving();  
        default oven:  
            oven.Cook();  
    }  
}  
  
//Correct usage by Liskov  
class Chef {  
    public void Cook(Oven oven){  
        oven.Cook();  
    }  
}
```



SOLID++: Interface Segregation

Clients should not be forced to depend upon interfaces that they do not use.

The goal is to reduce the side effects and the amount of changes needed in a system by splitting the software into multiple, smaller and independent parts grouped by functionality. This will **enforce modularity and composition of behaviours**.



SOLID++: Interface Segregation

```
public class Car : IAmACar { ... }

public interface IAmACar{
    void GoTo(Location location);
    void RefillGasoline(int gallons);
    int CurrentMileage();
}
```



SOLID++: Interface Segregation

```
public class Car : IAmACar { ... }

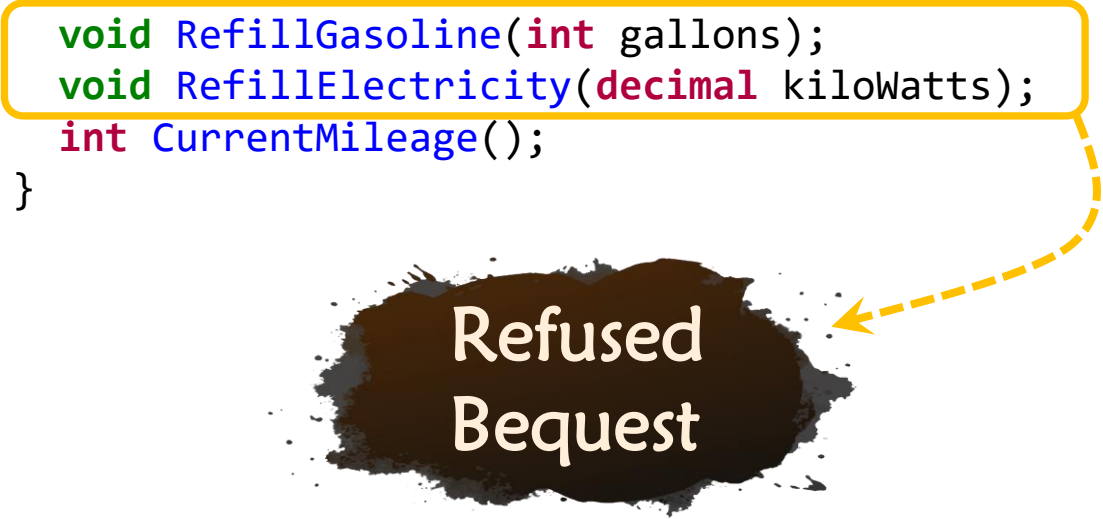
public interface IAmACar{
    void GoTo(Location location);
    void RefillGasoline(int gallons);
    void RefillElectricity(decimal kilowatts);
    int CurrentMileage();
}
```



SOLID++: Interface Segregation

```
public class Car : IAmACar { ... }
```

```
public interface IAmACar{  
    void GoTo(Location location);  
    void RefillGasoline(int gallons);  
    void RefillElectricity(decimal kiloWatts);  
    int CurrentMileage();  
}
```



Refused
Bequest



SOLID++: Interface Segregation

```
public interface IAmACar{  
    void GoTo(Location location);  
    int CurrentMileage();  
}
```

```
public interface IUseGasoline : IAmACar{  
    void RefillGasoline(int gallons);  
}
```

```
public interface IUseElectricity : IAmACar{  
    void RefillElectricity(decimal kiloWatts);  
}
```



SOLID++: Dependency Inversion

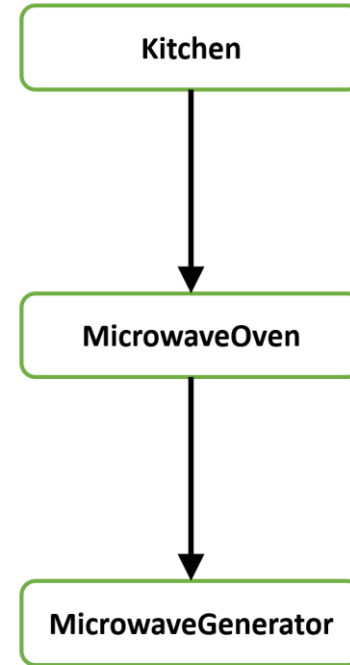
- High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions

Both high-level and low-level modules depend on the abstraction. The design principle does not just change the direction of the dependency, as you might have expected from its name. It splits the dependency between the high-level and low-level modules by *introducing an abstraction (contract) between* them.



SOLID++: Dependency Inversion

```
public class Kitchen{  
    private MicrowaveOven _oven;  
    public Kitchen(){  
        _oven = new MicrowaveOven();  
    }  
    ...  
}  
  
public class MicrowaveOven{  
    private MicrowaveGenerator _heater;  
  
    public Microwave(){  
        _heater = new MicrowaveGenerator();  
    }  
    ...  
}  
  
public class MicrowaveGenerator{...}
```



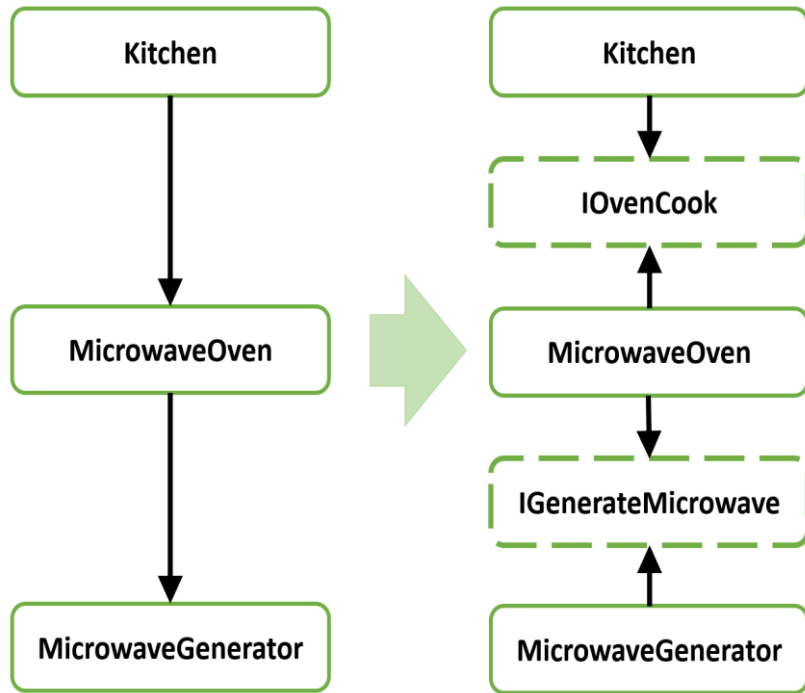
SOLID++: Dependency Inversion

```
public interface IOvenCook{ ... }
public interface IGenerateMicrowaves{ ... }

public class Kitchen{
    private IOvenCook _oven;
    public Kitchen(IOvenCook oven){
        _oven = oven;
    }
    ...
}

public class MicrowaveOven : IOvenCook{
    private IGenerateMicrowaves _heater;
    public Microwave(IGenerateMicrowaves heater){
        _heater = heater;
    }
    ...
}

public class MicrowaveGenerator : IGenerateMicrowaves
{ ... }
```



SOLID++: Balanced Abstraction

All code constructs grouped by a higher-level construct should be at the same level of abstraction.

- All instructions inside a method should be at the same level of abstraction
- All public methods inside a class should be at the same level of abstraction
- All classes should be inside a package/namespace
- All sibling packages/namespace should be inside a parent package/namespace.



SOLID++: Balanced Abstraction

```
public class Car {  
    public int CurrentMileage(){...}  
    public void TravelTo(Location location){...}  
    public void Save(){...}  
}
```

```
public class LeapYear {  
    public bool IsLeapYear(int year){  
        if((year % 4 == 0) && !year.IsMultipleOf(100))  
        {  
            ...  
        }  
        ...  
    }  
}
```



SOLID++: Least Astonishment (a.k.a. WTF)

People are part of the system. The design should match the user's experience, expectations, and mental models.

We are software developers, so the principle says that we shouldn't mislead the developers who will work on the code in the future with names that are not coherent with the behaviour inside the constructs.



SOLID++: Least Astonishment (a.k.a. WTF)

```
int Multiply(int a, int b)
{
    return a + b;
}

int SaveFile(string filename, string text)
{
    Console.WriteLine(text);
}

public class SomethingFactory {
    public void DoSomething()
    {
        //write on Database
    }
}
```

